

Spring is not magic · part 1 of 2

How a bean is born

Scanning, injection, scopes, lifecycle — the whole path from @Component to your object

`github.com/ELMAALMIA/minispring`

Ayoub EL Maalmi · Java & Spring Boot · 45 minutes

Where we are going

Six steps, from what Spring is to a fully built bean

- 1 Spring, then Spring Boot**
WAR, JAR, who owns the server
- 2 Why injection at all**
who chooses the implementation
- 3 What an annotation really is**
bytecode, javap, reflection
- 4 Who reads which annotation**
minispring, Spring, and XML
- 5 Creating the bean**
constructor, scopes, lifecycle, cycles
- 6 Takeaways**
and what part 2 covers

You have written this a thousand times

```
@Service
public class OrderService {

    @Autowired
    private OrderRepository
repository;
}
```

Who creates this object?

And how does it know what to put inside?

Today we read the answer, line by line. It is not magic, it is about 1,200 lines.



1

Spring, then Spring Boot

What actually changed between the two

Twenty years, one model

The source of metadata changed. The model never did.

2003

Spring 1.0

XML: `<bean>`

2007

Spring 2.5

`@Component,`
`@Autowired`

2009

Spring 3.0

`@Configuration,`
`@Bean`

2014

Spring Boot

`auto-configuration`

2022

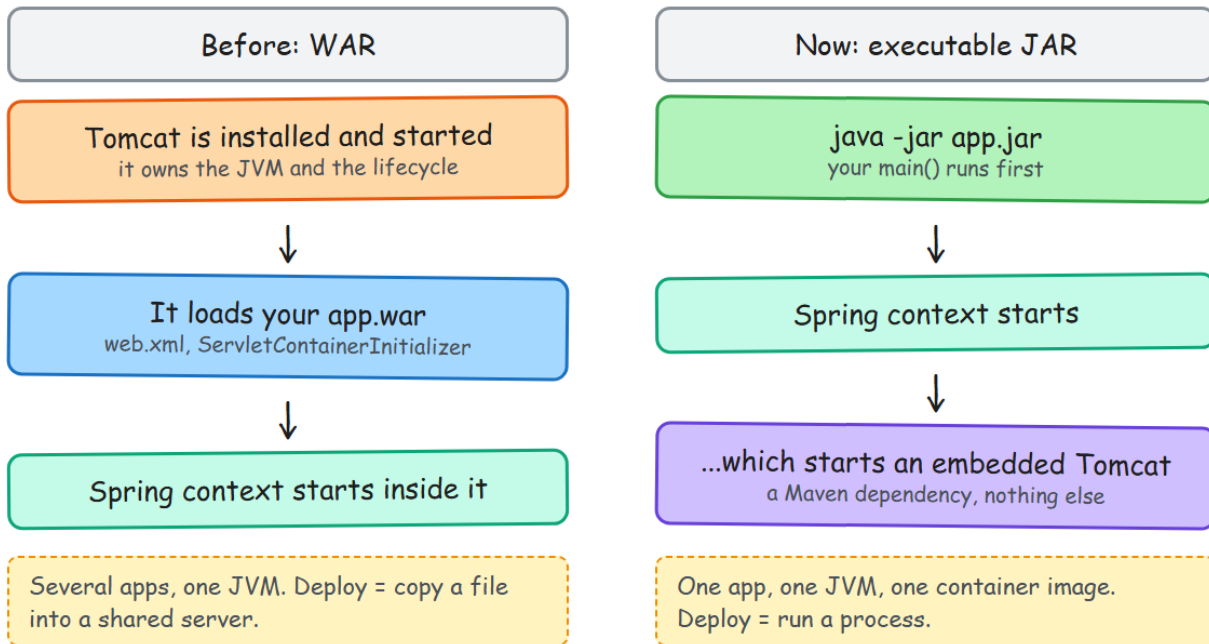
Spring 6

`AOT, native images`

Whatever the source — XML, annotations, Java, auto-configuration — the container sees one thing:
a `BeanDefinition`.

WAR or executable JAR: who owns the server?

The change Spring Boot really made





2

Why injection at all

The problem before the solution

Before the container: you wire it yourself

Everybody has written this

```
var repository = new InMemoryOrderRepository();  
var audit      = new AuditService();  
var service    = new OrderServiceImpl(repository, audit, publisher);  
var controller = new OrderController(service);
```

- It works — until the graph has forty objects instead of four
- Every caller has to know the whole graph, just to build one object
- Swapping an implementation means editing every place that says new
- And someone has to decide who gets closed, in which order

The problem was never typing new. It is that the caller has to know everything.

What you actually gain

It is not fewer lines. It is who chooses.

```
@Component
class OrderServiceImpl implements OrderService {
    OrderServiceImpl(OrderRepository repository, AuditService audit) { ... }
}

try (var context = AnnotationApplicationContext.scan("com.example")) {
    context.getBean(OrderController.class);    // the graph is built for you
}
```

- The class still asks for what it needs: nothing is hidden, it is all in the constructor
- But it no longer chooses which implementation it gets — that is the inversion of control
- So a test builds it with a fake repository, in one line, with no framework at all

Dependency injection is a design decision. The container is only the tool that carries it out.

A container is a Map, plus rules

```
Map<String, Object>
```

The interesting part is not the Map. It is the rules that fill it:

- Find the classes to manage
- Describe them before creating them
- Create each one after its dependencies
- Run lifecycle callbacks at the right moment
- Let something wrap a bean before it is handed out (part 2)
- Decide whether a bean should exist at all (part 2)



3

What an annotation really is

The part nobody explains, and everything depends on

An annotation is an interface with no code

The real file from the project

```
@Documented
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.TYPE)
public @interface Component {

    String value() default "";
}
```

Six lines.

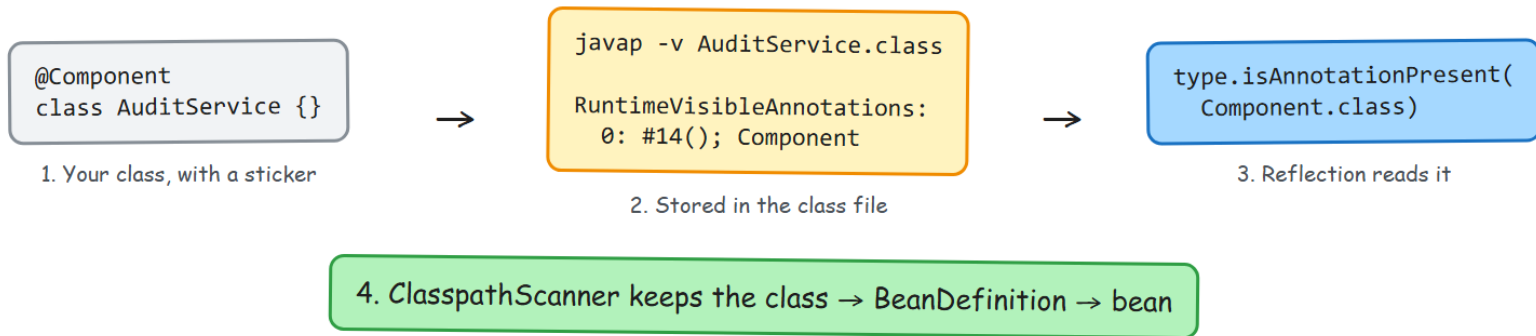
No code runs. No class is created. It only says: someone might be interested in this class.

The interesting part is the two lines above it.

@interface, not interface. The compiler turns it into metadata stored in the class file.

An annotation does nothing. Something has to read it.

@Component is a sticker · reflection is the reader · the container acts



Change RUNTIME to CLASS and the annotation is dropped from the class file: the scanner finds nothing, silently

Proof: javap, and one word changed

The demo that makes it click

```
$ javap -v AuditService.class
```

```
RuntimeVisibleAnnotations:
```

```
0: #14()
```

```
io.minispring.container.annotation.Component
```

RUNTIME

```
$ javap -v AuditService.class
```

```
RuntimeInvisibleAnnotations:
```

```
0: #14()
```

```
io.minispring.container.annotation.Component
```

CLASS — the default

One word, and the scanner finds nothing — with no error message. Everyone hits this once.

The three retentions, and who uses them

Retention	Lives until	Read by	Example
SOURCE	the compiler	annotation processors	@Override, Lombok
CLASS	the class file	bytecode tools	nullability annotations
RUNTIME	for ever	reflection, at run time	@Component, @Autowired

@Target says where the sticker may be put: a class, a method, a parameter. The compiler enforces it, nothing else does.

A framework that reads annotations at run time needs RUNTIME. That is the whole rule.

The reflection API, as used by the container

Six calls. That is the whole toolbox.

The call	What it does	Where in minispring
<code>Class.forName(n, false, cl)</code>	load a class without initialising it	<code>ClasspathScanner</code>
<code>type.isAnnotationPresent(..)</code>	read the sticker	<code>ClasspathScanner</code>
<code>type.getDeclaredConstructors()</code>	find the constructor to use	<code>BeanDefinitionReader</code>
<code>constructor.newInstance(args)</code>	create the bean	<code>BeanFactory</code>
<code>method.invoke(bean)</code>	run <code>@PostConstruct</code>	<code>BeanFactory</code>
<code>Proxy.newProxyInstance(..)</code>	wrap the bean (part 2)	<code>TransactionalProcessor</code>

If you know these six calls, you can write a dependency injection container.



4

Who reads which annotation

minispring, Spring, and the XML we used to write

Who reads what

Same model, three ways to feed it

Annotation	Read by, in minispring	The Spring class	In XML, before
@Component	ClasspathScanner	ClassPathBeanDefinitionScanner	<context:component-scan>
@Autowired	BeanDefinitionReader	AutowiredAnnotation...Processor	<constructor-arg ref>
@Qualifier @Primary	DependencyResolver	QualifierAnnotation...Resolver	<qualifier>, primary=
@Scope	BeanDefinitionReader	ScopeMetadataResolver	scope="prototype"
@PostConstruct @PreDestroy	LifecycleMethods	CommonAnnotation...Processor	init-method=, destroy-method=
@Configuration @Bean	BeanDefinitionReader	ConfigurationClassPostProcessor	factory-bean, factory-method

The annotation is never the mechanism. It is one of three ways to write the same BeanDefinition.

The same bean, written three ways

Different syntax, identical BeanDefinition

2008 — XML

```
<bean id="orderService"  
      class="shop.OrderServiceImpl">  
  <constructor-arg ref="orderRepository"/>  
</bean>  
  
<bean id="orderRepository"  
      class="shop.InMemoryOrderRepository"  
      init-method="open"  
      destroy-method="close"  
      scope="singleton"/>
```

Today — annotations

```
@Component  
class OrderServiceImpl implements OrderService {  
    OrderServiceImpl(OrderRepository repo) { }  
}
```

Today — Java configuration

```
@Configuration  
class AppConfig {  
    @Bean  
    OrderService orderService(OrderRepository r) {  
        return new OrderServiceImpl(r);  
    }  
}
```

The container reads all three into the same object. XML did not disappear because it was wrong, but because the metadata sat far from the code.

Zoom 1 — finding the classes

Java has no API to list the classes of a package

```
private static boolean isInstantiableComponent(Class<?> type) {  
    int modifiers = type.getModifiers();  
    return isComponent(type)  
        && !type.isInterface()  
        && !Modifier.isAbstract(modifiers)  
        && (!type.isMemberClass() || Modifier.isStatic(modifiers));  
}
```

- Ask the ClassLoader for the package directory, then walk it with Files.walk
- Load with Class.forName(name, false, loader): finding must not run static initialisers
- Sort the result, so that startup order never depends on the file system

Zoom 2 — choosing a constructor, then a bean

Two rules, and an error message that does the teaching

- One constructor: use it, no annotation needed — that is why `@Autowired` disappeared from modern Spring code
- Several: use the one marked `@Autowired`; several marked, or none: fail, and list the candidates
- Then, per parameter: type match, then `@Qualifier` by name, then `@Primary`. Never a guess.

```
2 beans of type com.example.PaymentGateway match, required by  
parameter 'gateway' of bean 'checkout': [stripeGateway, paypalGateway].  
Mark one with @Primary, or choose one with @Qualifier("name").
```

An error message that names the parameter and every candidate is worth more than a page of documentation.

Scopes: how many instances exist

One shared, or one per request

```
@Component                                // one instance, created at startup
class Cache { }

@Component @Scope(ScopeType.PROTOTYPE)    // a new one on every lookup
class ShoppingCart { }
```

- Singletons are created at startup, so a wiring mistake fails then — not at three in the morning
- A prototype is created on every request, and the container keeps no reference to it
- Which is why a prototype never receives `@PreDestroy`: nobody is holding it for you

A prototype injected into a singleton is created once, and once only. Same in Spring, and it surprises everyone.

The lifecycle: after injection, before the end

Order is the whole feature

```
init service with repository true      <- @PostConstruct, after injection  
  
destroy controller                     <- close(), in reverse creation order  
destroy service  
destroy repository
```

- `@PostConstruct` runs after injection, so the dependencies are usable — that is its only purpose
- `close()` destroys in reverse creation order, so a bean dies before what it depends on
- A failing `@PreDestroy` is logged, and the other beans are still closed
- If startup fails halfway, what was already created is destroyed before the error propagates

Circular dependencies: why a constructor cannot

And why that is good news

```
class Chicken { Chicken(Egg egg) { } }  
class Egg     { Egg(Chicken chicken) { } }
```

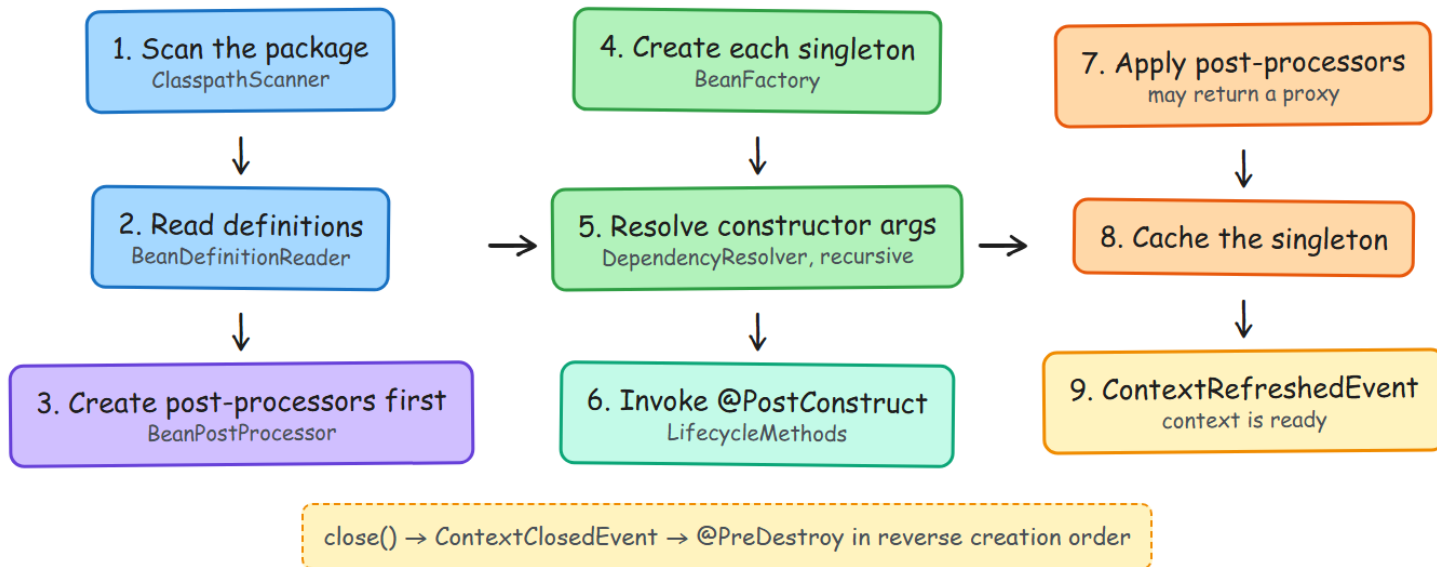
Circular dependency: rock -> paper -> scissors -> rock. Constructor injection cannot create a bean before its own dependencies exist; move the shared logic into a separate bean to break the cycle.

- With constructors it is simply impossible: A cannot exist before B if each needs the other
- Spring resolves cycles only for field injection, by handing out a half-built object from an early cache
- So a cycle is a design problem, and constructor injection is what makes it visible

The container refuses, and shows the whole chain. That is a feature, not a limitation.

What happens when the context starts

`AnnotationApplicationContext.scan("com.example")`



Three things to take home

1

An annotation does nothing

Something has to read it. javap showed you exactly what that something reads.

2

A container is a Map and a few rules

Definitions first, instances second. Every feature follows that order.

3

Constructor injection is a design choice

It makes your dependencies visible, and your cycles impossible to ignore.

`github.com/ELMAALMIA/minispring`

Part 2: why your `@Transactional` sometimes does nothing, and how `@Conditional` decides.